

HELLO!

I am Sayeed Unnisa

I am here because I love to give presentations on **Java Full Stack**

You can find me at sayeedunnisa.js@gmail.com

SPRING MVC



Spring Boot Web MVC

- Spring Boot MVC is a module of the spring framework which deals with the MVC pattern or model view controller
 - It is used to develop web application in standard.
 - By using this we can de-couple Request processing logic and Display code
 - Servlets/JSP contains Processing code and Display code combinedly.
 - If we move from one UI(View) to another UI(View) , then Controller code remains same.
 - By using Browser / Web Browser user can make request
1. Enter URL in Addressbar [GET]
 2. Click on HyperLinks (<a> tags) [GET]
 3. Submit HTML Forms (GET/POST)

Spring Boot Web MVC

MVC : Design Pattern.

M = Model (Data/Object holds data)

V = View (UI/HTML/JSP/JSF..) [Display Result]

C = Controller (Class) [Request Processing Logic]

- DispatcherServlet is a pre-defined Servlet, given by Spring Web F/w.
- It behaves like FrontController.
- Actual Request Processing Logic exist inside Controller Methods.
- Browser request contains URL(PATH) and HttpMethod ..etc
- Based on these details FrontController has to identify controller #method details. So, it uses HandlerMapper(Register)

HandlerMapper

KEY	VALUE
PATH + HttpMethod	Controller#method
/emp/save + POST	EmployeeController#saveEmp()
/user/login + GET	UserController#showLogin()

- HandlerMapper is a Map/Register contains details like
"For which request where to go"
- FrontController will call/execute Controller#method which process the request and returns ViewName (ex: Home, Login, Profile) without any extension and location details and also data if exist stored in Model (K=V)
- FrontController takes help of ViewResolver that gives File Location (prefix) and file extension (suffix) and finally returns ViewPage
- ViewPage = prefix + viewName + suffix
= /mypages/ Home .jsp

- If we move from one UI/View technology to another one then Controller code remains same.
- Only View Resolver details and Views need to be modified
- Model means sharing data to View/UI Page in KEY=VAL format.
- This is Dynamic Data.
- View is used to display content to End-User (Form, Table, Image, Text..etc) we need to define inside View (HTML Page).
- This view must be generated by Server as Response which contains HTML/CSS/JS.
- Data Rendering means replace Expressions with actual data

Spring Boot :

1. Provides Embeeded Server (No Download + No Install)
Apache Tomcat**, JBoss Undertow, Eclipse Jetty.
2. Default port number is 8080
`server.port=9090` in properties file
3. Package Format must be .war (Web Archive).
but we can even choose .jar or .war If we choose "Embeeded Server".
4. Comes with AutoConfiguration of DispatcherServlet (no web.xml),
HandlerMapper and ViewResolver.

5. We must define Controller , View Page and Properties file [contains prefix and suffix].
6. Spring Web Dependency must be added while creating application.
7. Must restart server (Stop Application and Start again) if we do any code modification.
8. We can not run application multiple times with same port number.
9. To work with JSP View, we need to add: Tomcat-Embedded-JASPER

- Controller class name can be any name but must be added with `@Controller` annotation.
- `@RequestMapping` links one PATH and HTTP method with Controller class method.
- URL/PATH is case sensitive : `/show`, `/SHOW`, `/Show` are different.
- Even File Name (View Page name) also. `HOME`, `home`, `Home` are different.
- `/` means root folder, Here root folder is 'webapp'

Request can be made in 3 ways:

1. Enter URL In addressbar [GET]
2. Click on HyperLink [GET]
 <a> tags / Links / Menu bar /
3. Submit HTML Form (GET/POST)

Http Method

GET : Fetch Resource from server to client

POST: Send/create new Resource from client to server

- **Resource** : file/ text / document/ image/ DB table row..etc
- GET with Parameters are used to do a task we are passing inputs.
- GET parameters are never stored at server

@Controller :

- Is a Steretype annotation
- It will help container to create object and also support HTTP Protocol

@RequestMapping :

- Link one PATH(URL) with httpMethod to a Controller#Method

Syntax:

`@RequestMapping(value="/path", method=RequestMethod.<TYPE>)`

Controller class :

- Code is written by developer
- Object created by container
- Method called by FrontController

Model:

- Model (l) is a Map type internally, which is used to hold data and share to View / UI page.
- It is created/altered(size increase/reduce) by Spring container.
- We need to use it as method parameter and call method `addAttribute(String key , Object value)`
- KEY is case-sensitive. Same key name must be used at UI using EL Expression Language : `${key}`
- DevTools is used to avoid server restart for code changes in application.

Html forms / ModelAttribute

- To collect end-user data, we use HTML Forms (Collection of Inputs)
- Register Form, Login Form, Enquiry Form, Payment Form, Feedback Form..etc
- Spring Container converts Entire Form data into one Object,
- When we enter data and submit the Form. ie called as ModelAttribute.
 - Create Object to Bean/Model class
 - Read data (Form Data) (`request.getParameter`)
 - Parse data if required (`Integer.parseInt`)
 - Set data to object (`obj.set__`)

- Programmer need to define one Model / Bean class
 - 1 Form = 1 class
 - 1 Input = 1 variable with set/get
 - Input Name === variableName
- This Object name in container will be className first letter lowercase ie camelCaseRule.

To read this object at Controller#method:

Syntax

@ModelAttribute("objName") ClassName localVariable

Ex: @ModelAttribute("student") Student sob

Syntax#2

@ModelAttribute ClassName objName

Ex: @ModelAttribute Student student

Model (I) :

Send Data from Controller to View (UI)

@ModelAttribute(C) :

Send data from View(UI) using HTML Form to Controller

Request Patameters: URL?key=val&key=val&key=val ...

Passing input to Controller#methods (application) to execute a task.

- Form Data is stored at application, but parameters are not recommended.
- Parameters are for input to process.

- In Spring Boot WEB MVC to read parameters in WEB MVC, as method parameter.

`@RequestParam("key") DataType variableName`

- Do not recommended to use primitive types in application instead of int, use Integer

- Every Request Parameter exist in code, must be passed from Request URL.
because they are internally required=true.
- We can make them optional request parameters.
- Then default value is NULL (if we do not pass data)

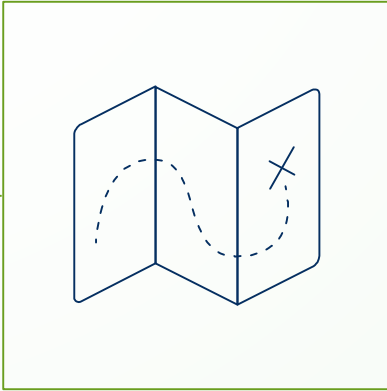
Syntax:

@RequestParam(value = "key", required = false) DataType variable

Redirect:

Dispatch from one controller method to another controller method for same request. i.e., execute multiple controller methods.

- Method `ReturnType` must be `String` and looks like:
`return "redirect:PATH"`
- Finally after executing chain of methods, the last method returns `ViewName` (UI Page Name) and given as `Response` to browser.
- If we use `redirect` concept, then always last method `response/model` is considered and sent to `ViewPage`, other methods `data/model` is local to that, can not be sent in response.
- **Model :** To send data from `Controller#method` to `View Page`
- **RedirectAttribute:** To send data from `Controller#method` to another `#method` while `redirect`.



THANKS!

Any questions?

You can find me at
sayeedunnisa.js@gmail.com